1. What is Gradient Descent?

Gradient Descent is an **iterative optimization algorithm** used to find the parameters θ (e.g., weights ***w*** and bias ***b*** in a model) that **minimize** a given loss function ***J(θ)***.

The main idea:

If you are on a curved surface (the loss function) and you want to reach the bottom (minimum), take steps in the direction of **steepest descent**, which is the negative of the gradient.

---

2. Mathematical Formulation

For parameters $\boldsymbol{\theta}$:

$$\theta := \theta - \alpha \cdot \nabla_\theta J(\theta)$$

Where:

- α = learning rate (step size)
- $\nabla_\theta J(\theta)$ = gradient of the loss function with respect to θ

Why negative gradient?

The gradient points in the direction of *steepest ascent*. To minimize, we go in the *opposite* direction.

---

3. The Gradient

If θ is a vector:

$$\nabla_\theta J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix}$$

Each partial derivative tells you how sensitive ***J*** is to small changes in that parameter.

4. Intuition behind the Math

Think of ***J(θ)*** as a **hilly landscape**:

- The gradient tells you the slope at your current point.
- If slope is large, you are far from the minimum → big correction.
- If slope is small, you are near the minimum → small correction.

Learning rate α

- Too large → overshoot, may diverge.
- Too small → very slow convergence.

---

5. Variants of Gradient Descent

- **Batch Gradient Descent** – uses all training data to compute gradient (stable but slow).
- **Stochastic Gradient Descent (SGD)** – uses 1 sample at a time (fast but noisy).
- **Mini-Batch Gradient Descent** – compromise: small batches (fast and more stable).

---

6. Example with Real Dataset (Linear Regression)

Model:

For samples (x_i, y_i):

$$\hat{y}_i = wx_i + b$$

where:

- w = slope (weight)
- b = intercept (bias)

Loss Function:

We use **Mean Squared Error (MSE)**:

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(y_i - (wx_i + b))^2$$

where m is the number of samples.

Gradients w.r.t *w* [for BGD (Batch Gradient Descent)]

$$\frac{\partial J}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{m}\sum_{i=1}^{m}(y_i - (wx_i + b))^2\right]$$

Using chain rule:

$$= \frac{1}{m}\sum_{i=1}^{m} 2\,(y_i - (wx_i + b)) \cdot \frac{\partial}{\partial w}\,[y_i - (wx_i + b)]$$

Since $\frac{\partial}{\partial w}[y_i - (wx_i + b)] = -x_i$:

$$\frac{\partial J}{\partial w} = -\frac{2}{m}\sum_{i=1}^{m}(y_i - (wx_i + b))\,x_i$$

Gradients w.r.t *b* [for BGD (Batch Gradient Descent)]

$$\frac{\partial J}{\partial b} = \frac{\partial}{\partial b}\left[\frac{1}{m}\sum_{i=1}^{m}\left(y_i - (wx_i + b)\right)^2\right]$$

Chain rule:

$$= \frac{1}{m}\sum_{i=1}^{m} 2\left(y_i - (wx_i + b)\right) \cdot \frac{\partial}{\partial b}\left[y_i - (wx_i + b)\right]$$

Since $\frac{\partial}{\partial b}[y_i - (wx_i + b)] = -1$:

$$\frac{\partial J}{\partial b} = -\frac{2}{m}\sum_{i=1}^{m}\left(y_i - (wx_i + b)\right)$$

Update rules:

$$w := w + \alpha \cdot \frac{2}{m}\sum_{i=1}^{m}\left(y_i - (wx_i + b)\right)x_i$$

$$b := b + \alpha \cdot \frac{2}{m}\sum_{i=1}^{m}\left(y_i - (wx_i + b)\right)$$

Dataset:

x	y
1	2
2	4
3	6

Initialization:

- $w = 0, b = 0$
- Learning rate $\alpha = 0.1$
- $m = 3$

Solve the above problem using BGD.

Iteration/epoch-1:

Predictions:

$$wx_i + b = [0, 0, 0]$$

Errors:

$$y_i - (wx_i + b) = [2, 4, 6]$$

Gradient w.r.t. w:

$$\frac{\partial J}{\partial w} = -\frac{2}{3}[(2)(1) + (4)(2) + (6)(3)] = -\frac{2}{3}(28) = -18.67$$

Gradient w.r.t. b:

$$\frac{\partial J}{\partial b} = -\frac{2}{3}[2 + 4 + 6] = -\frac{2}{3}(12) = -8$$

Update:

$$w := 0 + 0.1 \cdot 18.67 = 1.867$$

$$b := 0 + 0.1 \cdot 8 = 0.8$$

Iteration/epoch-2:

Predictions:

$$[1.867(1) + 0.8,\ 1.867(2) + 0.8,\ 1.867(3) + 0.8] = [2.667, 4.534, 6.401]$$

Errors:

$$[2 - 2.667,\ 4 - 4.534,\ 6 - 6.401] = [-0.667, -0.534, -0.401]$$

Gradient w.r.t. w:

$$\frac{\partial J}{\partial w} = -\frac{2}{3}[(-0.667)(1) + (-0.534)(2) + (-0.401)(3)]$$

$$= -\frac{2}{3}(-2.936) \approx 1.957$$

Gradient w.r.t. b:

$$\frac{\partial J}{\partial b} = -\frac{2}{3}(-1.602) \approx 1.068$$

Update:

$$w := 1.867 - 0.1 \cdot 1.957 \approx 1.671$$

$$b := 0.8 - 0.1 \cdot 1.068 \approx 0.693$$

Repeating brings $w \to 2.0$, $b \to 0$.

7. Key Takeaways

- Gradient descent is a **general** method for minimizing differentiable functions.
- Sigmoid/logistic regression, linear regression, neural networks — all rely on it.
- The choice of learning rate and stopping criteria is critical.
- The "gradient" is the **mathematical compass** pointing toward the minimum.

Solution using SGD (Stochastic Gradient Descent)

Setup

- Model: $\hat{y} = wx + b$
- Loss (per sample): $J_i = (y_i - (wx_i + b))^2$
- Single-sample gradients:

$$\frac{\partial J_i}{\partial w} = -2\,(y_i - (wx_i + b))\,x_i, \qquad \frac{\partial J_i}{\partial b} = -2\,(y_i - (wx_i + b))$$

- SGD update (step on one (x_i, y_i)):

$$w \leftarrow w - \alpha \frac{\partial J_i}{\partial w} = w + 2\alpha\,(y_i - (wx_i + b))\,x_i$$

$$b \leftarrow b - \alpha \frac{\partial J_i}{\partial b} = b + 2\alpha\,(y_i - (wx_i + b))$$

We'll use learning rate $\alpha = 0.1$, initialize $w = 0,\ b = 0$, and process samples in order $(1, 2) \to (2, 4) \to (3, 6)$.

Epoch 1 (SGD: update after each sample)

Sample 1: $(x_1, y_1) = (1, 2)$

- $\hat{y}_1 = 0 \cdot 1 + 0 = 0$
- Error $e_1 = y_1 - \hat{y}_1 = 2 - 0 = 2$
- Update:
  - $w \leftarrow 0 + 2(0.1)(2)(1) = 0.4$
  - $b \leftarrow 0 + 2(0.1)(2) = 0.4$

Sample 2: $(x_2, y_2) = (2, 4)$

- $\hat{y}_2 = 0.4 \cdot 2 + 0.4 = 1.2$
- $e_2 = 4 - 1.2 = 2.8$
- Update:
  - $w \leftarrow 0.4 + 2(0.1)(2.8)(2) = 1.52$
  - $b \leftarrow 0.4 + 2(0.1)(2.8) = 0.96$

Sample 3: $(x_3, y_3) = (3, 6)$

- $\hat{y}_3 = 1.52 \cdot 3 + 0.96 = 5.52$
- $e_3 = 6 - 5.52 = 0.48$
- Update:
  - $w \leftarrow 1.52 + 2(0.1)(0.48)(3) = 1.808$
  - $b \leftarrow 0.96 + 2(0.1)(0.48) = 1.056$

After Epoch 1: $w = 1.808,\ b = 1.056$

Epoch 2 (same order)

Sample 1: $(1, 2)$

- $\hat{y}_1 = 1.808 \cdot 1 + 1.056 = 2.864$
- $e_1 = 2 - 2.864 = -0.864$
- Update:
  - $w \leftarrow 1.808 + 2(0.1)(-0.864)(1) = 1.6352$
  - $b \leftarrow 1.056 + 2(0.1)(-0.864) = 0.8832$

Sample 2: $(2, 4)$

- $\hat{y}_2 = 1.6352 \cdot 2 + 0.8832 = 4.1536$
- $e_2 = 4 - 4.1536 = -0.1536$
- Update:
  - $w \leftarrow 1.6352 + 2(0.1)(-0.1536)(2) = 1.57376$
  - $b \leftarrow 0.8832 + 2(0.1)(-0.1536) = 0.85248$

Sample 3: $(3, 6)$

- $\hat{y}_3 = 1.57376 \cdot 3 + 0.85248 = 5.57376$
- $e_3 = 6 - 5.57376 = 0.42624$
- Update:
  - $w \leftarrow 1.57376 + 2(0.1)(0.42624)(3) = 1.829504$
  - $b \leftarrow 0.85248 + 2(0.1)(0.42624) = 0.937728$

After Epoch 2: $w \approx 1.829504,\ b \approx 0.937728$

Solution using mini-BGD

Here's **mini-batch gradient descent (mini-BGD)** on your 3-point dataset using **batch size** $k = 2$, learning rate $\alpha = 0.1$, and the "error" form $(y - (wx + b))$. We update **once per mini-batch**.

Model: $\hat{y} = wx + b$
Loss (MSE): $J(w, b) = \frac{1}{m} \sum_{i=1}^{m} \left(y_i - (wx_i + b)\right)^2$
Mini-batch gradients (batch B of size k):

$$\frac{\partial J_B}{\partial w} = -\frac{2}{k} \sum_{i \in B} (y_i - (wx_i + b))\, x_i, \quad \frac{\partial J_B}{\partial b} = -\frac{2}{k} \sum_{i \in B} (y_i - (wx_i + b))$$

Updates (equivalently using $(y - \hat{y})$ directly):

$$w \leftarrow w + \frac{2\alpha}{k} \sum_{i \in B} (y_i - (wx_i + b))\, x_i, \qquad b \leftarrow b + \frac{2\alpha}{k} \sum_{i \in B} (y_i - (wx_i + b))$$

Data (order fixed): $(1, 2), (2, 4), (3, 6)$.
Batches per epoch: $B_1 = \{(1, 2), (2, 4)\}$ (size 2), $B_2 = \{(3, 6)\}$ (size 1).
Init: $w_0 = 0,\ b_0 = 0$.

Epoch 1

Batch $B_1 = \{(1, 2), (2, 4)\}$, $k = 2$

- Current $w = 0,\ b = 0 \rightarrow$ predictions $[0, 0]$, errors $e = [2, 4]$
- Sufficient sums: $\sum ex = 2 \cdot 1 + 4 \cdot 2 = 10$, $\sum e = 2 + 4 = 6$
- Update factor: $\frac{2\alpha}{k} = \frac{2 \cdot 0.1}{2} = 0.1$
- **Update:** $w \leftarrow 0 + 0.1 \cdot 10 = 1.0, \quad b \leftarrow 0 + 0.1 \cdot 6 = 0.6$

Batch $B_2 = \{(3, 6)\}$, $k = 1$

- Now $w = 1.0,\ b = 0.6$; prediction at $x = 3$: $\hat{y} = 3.6$; error $e = 6 - 3.6 = 2.4$
- Sums: $\sum ex = 2.4 \cdot 3 = 7.2,\ \sum e = 2.4$
- Update factor: $\frac{2\alpha}{k} = 0.2$
- **Update:** $w \leftarrow 1.0 + 0.2 \cdot 7.2 = 2.44, \quad b \leftarrow 0.6 + 0.2 \cdot 2.4 = 1.08$

After Epoch 1: $w = 2.44,\ b = 1.08$
(Optional) Dataset MSE: $J = \frac{(-1.52)^2 + (-1.96)^2 + (-2.4)^2}{3} \approx 3.971$

Epoch 2

Batch $B_1 = \{(1,2),(2,4)\}, k = 2$

- Current $w = 2.44,\ b = 1.08$
- Predictions at $x = 1, 2$: $[3.52,\ 5.96]$; errors $e = [2 - 3.52 = -1.52,\ 4 - 5.96 = -1.96]$
- Sums: $\sum ex = (-1.52) \cdot 1 + (-1.96) \cdot 2 = -5.44,\ \sum e = -3.48$
- Factor: 0.1
- **Update:** $w \leftarrow 2.44 + 0.1 \cdot (-5.44) = 1.896, \quad b \leftarrow 1.08 + 0.1 \cdot (-3.48) = 0.732$

Batch $B_2 = \{(3,6)\}, k = 1$

- Now $w = 1.896,\ b = 0.732$
- Prediction at $x = 3$: $\hat{y} = 1.896 \cdot 3 + 0.732 = 6.42$; error $e = 6 - 6.42 = -0.42$
- Sums: $\sum ex = (-0.42) \cdot 3 = -1.26,\ \sum e = -0.42$
- Factor: 0.2
- **Update:** $w \leftarrow 1.896 + 0.2 \cdot (-1.26) = 1.644, \quad b \leftarrow 0.732 + 0.2 \cdot (-0.42) = 0.648$

After Epoch 2: $w = 1.644,\ b = 0.648$

(Optional) Dataset MSE: ≈ 0.0886

Notes & Takeaways

- **Mini-BGD** computes gradients on small chunks → **more stable** than SGD, often **faster** than full BGD.
- With α=0.1 and our batching, Epoch 1 overshoots (high MSE), then Epoch 2 corrects sharply toward the true line y=2x (ideal w=2, b=0).
- Different **batch orders** (or shuffling each epoch) can change the path but typically converge to the same solution for convex problems like linear regression.